

ساختمان داده

از صفر تا درخت و گراف

تنظیم کننده: [سینا شجاعیان]

نام استاد: [دکتر اسدی]

عنوان درس: [ساختمان داده (داده و

ساختار)]

ترم / سال: [1405]

رشته و مقطع: [مهندسی کامپیوتر]

دانشگاه / دانشکده: [آزاد شیراز]

صف و صف حلقوی

فصل ۳

پشته (Stack)

فصل ۲

تحلیل الگوریتم

فصل ۱

درخت

فصل ۶

لیست پیوندی

فصل ۵

توابع بازگشتی

فصل ۴

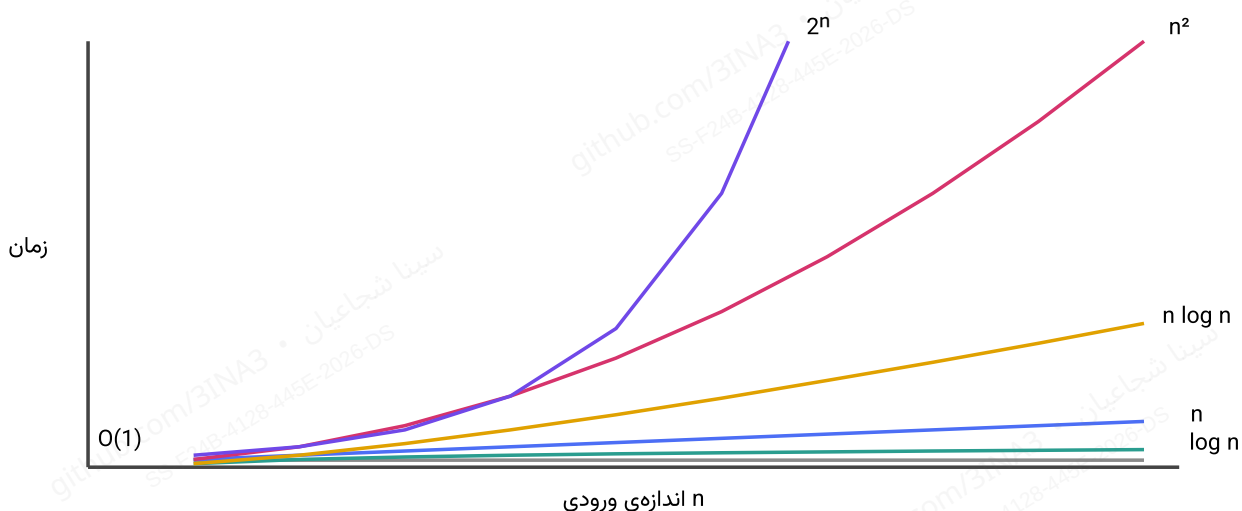
گراف

فصل ۷

۱. تحلیل الگوریتم

تحلیل یعنی سنجش زمان (تابع پیچیدگی $f(n)$) و حافظه. در مثال جزوه، حلقه‌ی تودرتو $f(n) = 2n^2 + n + 1$ می‌دهد (در جزوه به اشتباه «= 1» نوشته شده) و فقط بزرگ‌ترین توان مهم است: $O(n^2)$.

نماد	معنا	رابطه با $f(n)$
O (بزرگ‌او)	حد بالا	$f \leq c \cdot g$
Ω	حد پایین	$f \geq c \cdot g$
Θ	حد دقیق (وسط)	هر دو



مقایسه‌ی رشد توابع: هرچه منحنی تندتر، الگوریتم برای n بزرگ کندتر

مثال‌ها به ترتیب رشد

تابع	پیچیدگی	دلیل
worker	$O(1)$	دسترسی مستقیم با اندیس
sum_list	$O(n)$	یک پیمایش
binary_search	$O(\log n)$	هر بار نصف می‌شود
merge_sort	$O(n \log n)$	$\log n$ سطح $\times n$ کار در هر سطح

```
def sum_list(arr: list[int]) → int:
    """حافظه 0(1)، زمان 0(n): جمع عناصر"""
    total = 0
    for x in arr:
        total += x
    return total

def merge_sort(arr: list[int]) → list[int]:
    """0(n) حافظه، 0(n log n) مرتب‌سازی ادغامی: همیشه"""
    if len(arr) ≤ 1:
        return arr
    mid = len(arr) // 2
    left, right = merge_sort(arr[:mid]), merge_sort(arr[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] ≤ right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

if __name__ == "__main__":
    print(merge_sort([5, 2, 9, 1, 7])) # [1, 2, 5, 7, 9]
```

قدم به قدم: $f(n)$ چگونه ساخته می‌شود؟

برای هر دستور، «چند بار اجرا می‌شود» را بشمارید و جمع بزنید. در مثال جزوه: حلقه‌ی بیرونی n بار، حلقه‌ی درونی در هر دور n بار؛ داخل حلقه‌ی درونی دو `print` ($2n^2$ دستور)، `print(j)` هر دور بیرونی یک بار (n دستور) و `print(i+j)` یک بار (n دستور). پس $f(n) = 2n^2 + n + 1$. وقتی n بزرگ می‌شود، n و 1 نسبت به n^2 ناچیزند و ضریب ثابت هم حذف می‌شود؛ نتیجه $\Theta(n^2)$.

n	n	$n \log n$	n^2	2^n
10	10	≈ 33	100	1024
1000	1000	≈ 9966	10^6	خارج از تصور

حافظه: فقط به فضای اضافه نگاه می‌کنیم. `sum_list` فقط یک متغیر `total` دارد: $O(1)$. لیست‌های کمکی می‌سازد: $O(n)$.

Python

```
def worker(arr: list[int], index: int) → int:
    """ $O(1)$  → بستگی ندارد n دسترسی مستقیم به یک خانه: تعداد کار به"""
    return arr[index]

def linear_sum(arr: list[int]) → int:
    """ $O(n)$  → هر عنصر دقیقاً یک بار دیده می‌شود"""
    total = 0
    for element in arr:
        total += element
    return total

if __name__ == "__main__":
    data = [4, 8, 15, 16]
    print(worker(data, 2), linear_sum(data))    # 15 43
```

جست‌وجوی دودویی (روش حلقه)

شرط: آرایه مرتب باشد. وسط را نگاه می‌کنیم؛ اگر برابر بود تمام؛ اگر هدف بزرگ‌تر بود نیمه‌ی چپ را دور می‌ریزیم؛ وگرنه نیمه‌ی راست را. هر مرحله بازه نصف می‌شود، پس بعد از حدود $\log_2 n$ مرحله تمام می‌شود. برای ۱ میلیون عنصر فقط حدود ۲۰ مقایسه!

```
جست‌وجوی 7 در [10, 8, 7, 6, 4, 3, 1] low=0 high=6
mid=3 → 6 < 7 → low=4           [ . . . 7 8 10 ]
mid=5 → 8 > 7 → high=4          [ . . . 7 ]
mid=4 → 7 == 7 → 4 اندیس پیدا شد
```

```
def binary_search(arr: list[int], target: int) → int:
    """0(1) حافظه، 0(log n) یا 1- زمان target اندیس"""
    low, high = 0, len(arr) - 1
    while low ≤ high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

if __name__ == "__main__":
    nums = [1, 3, 4, 6, 7, 8, 10]
    print(binary_search(nums, 7), binary_search(nums, 5))    # 4 -1
```

مرتب‌سازی ادغامی: تقسیم، حل، ترکیب

لیست را مدام نصف می‌کنیم تا به لیست‌های یک‌عنصری برسیم (که ذاتاً مرتب‌اند)، بعد دو لیست مرتب را با دو اشاره‌گر i, j ادغام می‌کنیم. تعداد سطوح $\log n$ و ادغام هر سطح n کار دارد $\Rightarrow O(n \log n)$ در تمامی حالت‌ها (Best = Average = Worst).

```
[5 2 9 1]
 /      \
[5 2]    [9 1]      تقسیم
[5][2]   [9][1]
 \ /     \ /
[2 5]    [1 9]      ادغام
  \      /
  [1 2 5 9]
```

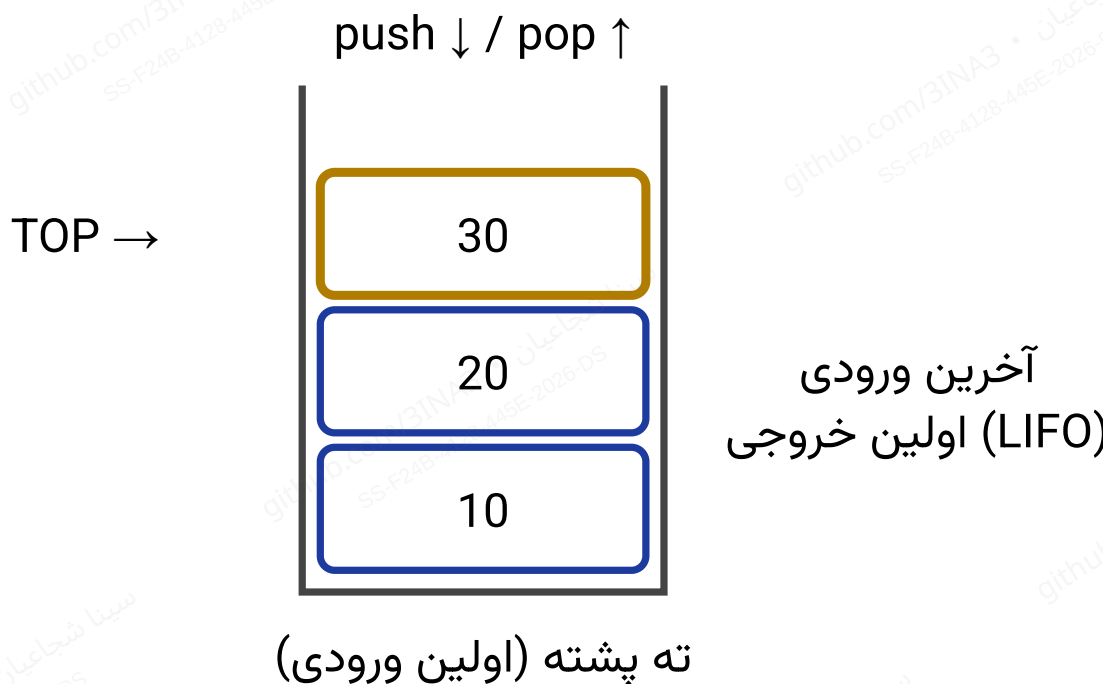
توضیح تکمیلی: نسخه‌ی درجای کلاس (با `merge_sort(arr)` بدون `return`) هم همین منطق را دارد:
`left_half = arr[:mid], right_half = arr[mid:]` و سه شمارنده‌ی i, j, k که نتیجه را در خود `arr` می‌نویسند.

```
def merge_sort_inplace(arr: list[int]) → None:
    """نسخه‌ی درجا مطابق کد کلاس: لیست ورودی را مرتب می‌کند"""
    if len(arr) > 1:
        mid = len(arr) // 2
        left_half, right_half = arr[:mid], arr[mid:]
        merge_sort_inplace(left_half)
        merge_sort_inplace(right_half)
        i = j = k = 0
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]; i += 1
            else:
                arr[k] = right_half[j]; j += 1
            k += 1
        while i < len(left_half):
            arr[k] = left_half[i]; i += 1; k += 1
        while j < len(right_half):
            arr[k] = right_half[j]; j += 1; k += 1

if __name__ == "__main__":
    a = [38, 27, 43, 3, 9, 82, 10]
    merge_sort_inplace(a)
    print(a)    # [3, 9, 10, 27, 38, 43, 82]
```

۲. پشته (Stack)

ساختاری **LIFO** (آخرین ورودی، اولین خروجی). فقط از «بالا» (TOP) کار می‌کنیم. شرط **push**: پر نبودن؛ شرط **pop**: خالی نبودن (وگرنه Underflow).



پشته: ورود و خروج فقط از بالا

push(1) push(2) push(3) pop() → 3

TOP → [3]

[2] [2] [2]

[1] [1] [1]

O(1): همه ی عملیات

```

class Stack:
    """پشته با ظرفیت محدود."""

    def __init__(self, limit: int = 100) → None:
        self.items: list = []
        self.limit = limit

    def is_empty(self) → bool:
        return not self.items

    def push(self, x) → bool:
        if len(self.items) ≥ self.limit:    # Overflow
            return False
        self.items.append(x)
        return True

    def pop(self):
        if self.is_empty():                # Underflow
            raise IndexError("stack is empty")
        return self.items.pop()

    def peek(self):
        if self.is_empty():
            raise IndexError("stack is empty")
        return self.items[-1]

    def find(self, x) → int:
        """اندیس عنصر (از ته پشته) یا -1"""
        return self.items.index(x) if x in self.items else -1

    def replace(self, old, new) → bool:
        if old not in self.items:
            return False
        self.items[self.items.index(old)] = new
        return True

    def show(self) → None:
        print(*reversed(self.items))      # به پایین TOP از

def d2b(n: int) → str:
    """تبدیل دهدهی به دودویی با پشته (باقیمانده‌ها برعکس خوانده می‌شوند)"""
    if n == 0:

```

```

    return "0"
s, out = Stack(), ""
while n:
    s.push(n % 2)
    n //= 2
while not s.is_empty():
    out += str(s.pop())
return out

if __name__ == "__main__":
    s = Stack(); [s.push(v) for v in (10, 20, 30)]
    s.show(); print(s.peek(), s.pop())    # 30 20 10 / 30 30
    print(d2b(13))                       # 1101

```

توضیح تکمیلی: نمادهای ریاضی: Prefix: +AB، Postfix: AB+، Infix: A+B، برای A+(B*C) → postfix: ABC*+، prefix: +A*BC (با جابه‌جایی عملگر به جای پرانتز).

چرا پشته مهم است؟ سه کاربرد

۱) دکمه‌ی Call Stack (۲، Undo) در بازگشت (فصل ۴)، تبدیل مبنا و بررسی پرانتزها. ایده‌ی مشترک: آخرین کاری که شروع شد، اولین کاری است که باید تمام شود.

d2b: ردگیری گام‌به‌گام

تبدیل ۱۳ به دودویی: باقی‌مانده‌های تقسیم بر ۲ از «کم‌ارزش» به «پرازش» تولید می‌شوند، اما خروجی باید از «پرازش» شروع شود. پشته دقیقاً ترتیب را برعکس می‌کند.

n	n % 2 (push)	n // 2	پشته (TOP سمت راست)
13	1	6	[1]
6	0	3	[1,0]
3	1	1	[1,0,1]
1	1	0	[1,0,1,1]

حالا چهار بار $1101 \Rightarrow 1, 0, 1, 1$. pop

معکوس کردن پشته فقط با پشته (revstack)

سه پشته‌ی پشت‌سرهم: انتقال از L به S1 ترتیب را برعکس می‌کند، از S1 به S2 دوباره درست می‌کند، و از S2 به L برعکس می‌کند؛ پس در نهایت L معکوس شده است.

Python

```
def revstack(L: "Stack") → None:
    """(L درجا روی) را با دو پشته‌ی کمکی معکوس می‌کند L محتوای"""
    s1, s2 = Stack(L.limit), Stack(L.limit)
    while not L.is_empty():
        s1.push(L.pop())
    while not s1.is_empty():
        s2.push(s1.pop())
    while not s2.is_empty():
        L.push(s2.pop())

def balanced(text: str) → bool:
    """توضیح تکمیلی: بررسی تعادل پرانتزها با پشته"""
    pairs, s = {"(": ")", "[": "]", "{": "}"}, Stack()
    for ch in text:
        if ch in "([{":
            s.push(ch)
        elif ch in pairs:
            if s.is_empty() or s.pop() != pairs[ch]:
                return False
    return s.is_empty()

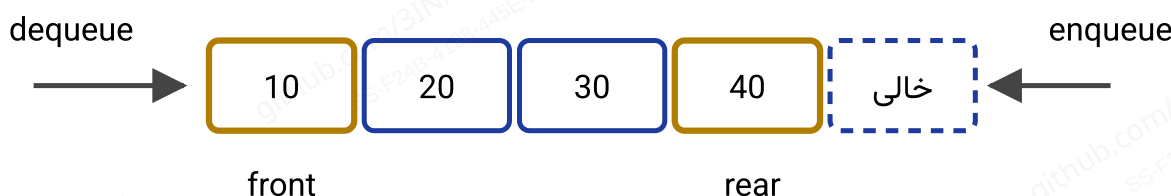
if __name__ == "__main__":
    st = Stack()
    for v in (1, 2, 3): st.push(v)
    revstack(st); st.show()          # 1 2 3 (TOP: 3 2 1 قبلاً از)
    print(balanced("[{()}]"), balanced("[]")) # True False
```

سریع‌ترین راه در پایتون: `lst.reverse()` یا `lst[::-1]`؛ اما هدف تمرین این است که با رفتار LIFO فکر کنید.

۳. صف و صف حلقوی

صف **FIFO** است: درج از `rear` ، حذف از `front` . در صف خطی با آرایه، با هر حذف یک خانه‌ی ابتدا هدر می‌رود؛ صف حلقوی با باقی‌مانده‌ی % این را حل می‌کند.

شرط	حالت صف حلقوی
<code>front == -1</code>	خالی
<code>(rear + 1) % limit == front</code>	پر
<code>front == rear</code>	تک‌عنصری



اول‌وارد، اول‌خارج: FIFO

صف خطی: درج از `rear` و حذف از `front`

```
اندیس: 0 1 2 3 4 5 6 7
مقدار: [50][ ][ ][ ][20][30][40]
        ↑front   ↑rear
rear = (1+7) % 8 = 0 ← بازگشت به ابتدا
```

```

class CQueue:
    """صف حلقوی با ظرفیت ثابت"""

    def __init__(self, limit: int = 10) → None:
        self.q = [None] * limit
        self.limit = limit
        self.front = self.rear = -1

    def is_empty(self) → bool:
        return self.front == -1

    def is_full(self) → bool:
        return (self.rear + 1) % self.limit == self.front

    def insert(self, x) → None:
        if self.is_full():
            raise OverflowError("queue is full")
        if self.is_empty():
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.limit
        self.q[self.rear] = x

    def delete(self):
        if self.is_empty():
            raise IndexError("queue is empty")
        x = self.q[self.front]
        if self.front == self.rear:          # تک‌عنصری → خالی
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.limit
        return x

    def display(self) → None:
        if self.is_empty():
            print("empty"); return
        i = self.front
        while True:
            print(self.q[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.limit
        print()

```

```
def reverse_stack(s: "Stack") → "Stack":
    """معکوس کردن پشته با صف (از فصل ۲)"""
    q, out = CQueue(), Stack()
    while not s.is_empty():
        q.insert(s.pop())
    while not q.is_empty():
        out.push(q.delete())
    return out

if __name__ == "__main__":
    q = CQueue(3)
    for v in (1, 2, 3): q.insert(v)
    q.delete(); q.insert(4); q.display()    # 2 3 4
```

صف خطی: اول مدل ذهنی، بعد کد

صف خطی را با آرایه‌ای به طول `limit` می‌سازیم و دو شاخص داریم: `front` (اولین عنصر) و `rear` (آخرین عنصر). هر دو در ابتدا -1 هستند (صف «آکبند/خالی» است). با اولین درج، `front` صفر می‌شود. با هر `dequeue` فقط `front` یکی جلو می‌رود؛ عنصر حذف‌شده در آرایه می‌ماند ولی دیگر جزو صف نیست.

شرط	حالت
<code>rear >= limit - 1</code>	پر
<code>front == -1</code>	خالی (روش اول)
<code>front > rear</code>	خالی (روش دوم، بعد از حذف‌ها)

```
limit=5          enqueue(10), (20), (30)      dequeue()→10
Q: [10][20][30][ ][ ]  front=0 rear=2          front=1 rear=2
                    ↑front   ↑rear              ↑front ↑rear    (خانه‌ی 0 هدر رفت)
```

```

class Queue:
    """صف خطی با آرایه (FIFO)."""

    def __init__(self, limit: int = 100) → None:
        self.q = [None] * limit
        self.limit = limit
        self.front = -1      # ابتدای صف
        self.rear = -1       # انتهای صف

    def is_full(self) → bool:
        return self.rear ≥ self.limit - 1

    def is_empty(self) → bool:
        return self.front == -1 or self.front > self.rear

    def enqueue(self, data) → None:
        if self.is_full():
            raise OverflowError("queue is full")
        if self.front == -1:      # اولین عنصر
            self.front = 0
        self.rear += 1
        self.q[self.rear] = data

    def dequeue(self):
        if self.is_empty():
            raise IndexError("queue is empty")
        x = self.q[self.front]
        self.front += 1
        return x

    def show(self) → None:
        if self.is_empty():
            print("empty")
            return
        print(*self.q[self.front:self.rear + 1])

    def replace(self, old, new) → bool:
        """عوض می‌کند new درون صف را با اولین old."""
        if self.is_empty():
            return False
        for i in range(self.front, self.rear + 1):
            if self.q[i] == old:
                self.q[i] = new

```

```
        return True
    return False
```

```
if __name__ == "__main__":
    q = Queue(5)
    for v in (10, 20, 30): q.enqueue(v)
    print(q.dequeue()); q.replace(30, 99); q.show()    # 10 / 20 99
```

معکوس کردن صف با پشته

صف را به پشته می‌ریزیم (اولین عنصر ته پشته می‌رود)، بعد پشته را به صف جدید خالی می‌کنیم؛ آخرین عنصر اول خارج می‌شود، پس ترتیب برعکس است.

Python

```
def reverse_queue(q: "CQueue") → "CQueue":
    """صف حلقوی را با پشته معکوس می‌کند."""
    s, new_q = Stack(), CQueue(q.limit)
    while not q.is_empty():
        s.push(q.delete())
    while not s.is_empty():
        new_q.insert(s.pop())
    return new_q

if __name__ == "__main__":
    q = CQueue(5)
    for v in (1, 2, 3): q.insert(v)
    reverse_queue(q).display()    # 3 2 1
```

۴. توابع بازگشتی

هر تابع بازگشتی سه جزء دارد: **شرط بازگشت (Base Case)**، **مقدار بازگشت** و **رابطه‌ی بازگشتی**. هر فراخوانی در Call Stack یک قاب می‌گیرد، پس حافظه‌ی عمق بازگشت مهم است (پیش‌فرض Python حدود ۱۰۰۰).

تابع	شرط / مقدار	رابطه	زمان
فاکتوریل	$n \leq 1 \rightarrow 1$	$n! = n \cdot (n-1)!$	$O(n)$
فیبوناچی	$n < 2 \rightarrow n$	$f(n) = f(n-1) + f(n-2)$	$O(2^n)$
جمع $a+b$	$b=0 \rightarrow a$	$S(a,b) = S(a,b-1) + 1$	$O(b)$
ضرب $a \times b$	$b=0 \rightarrow 0$	$M(a,b) = M(a,b-1) + a$	$O(b)$
تقسیم $a \div b$	$a < b \rightarrow 0$	$D(a,b) = D(a-b,b) + 1$	$O(a/b)$
پالیندروم	$len \leq 1 \rightarrow \text{True}$	سر=ته و بررسی وسط	$O(n)$
برج هانوی	$n=1 \rightarrow \text{یک حرکت}$	$T(n) = 2T(n-1) + 1$	$O(2^n)$
جست‌وجوی دودویی	$first > last \rightarrow -1$	نصف‌کردن بازه	$O(\log n)$

```

def fact(n: int) → int:
    return 1 if n ≤ 1 else n * fact(n - 1)

def fib(n: int) → int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)

def add(a: int, b: int) → int:          # b ≥ 0
    return a if b == 0 else add(a, b - 1) + 1

def mul(a: int, b: int) → int:
    return 0 if b == 0 else mul(a, b - 1) + a

def div(a: int, b: int) → int:          # خارج قسمت صحیح، b > 0
    return 0 if a < b else div(a - b, b) + 1

def is_palindrome(s: str) → bool:
    if len(s) ≤ 1:
        return True
    return s[0] == s[-1] and is_palindrome(s[1:-1])

def hanoi(n: int, src: str, aux: str, dst: str) → None:
    """A→C با کمک B2 تعداد حرکت‌ها ^n - 1."""
    if n == 1:
        print(src, "to", dst)
        return
    hanoi(n - 1, src, dst, aux)
    print(src, "to", dst)
    hanoi(n - 1, aux, src, dst)

def bsearch(arr: list[int], x: int, first: int = 0, last: int | None = None) → int:
    """یا 1- x آرایه‌ی مرتب؛ اندیس"""
    if last is None:
        last = len(arr) - 1
    if first > last:
        return -1
    mid = (first + last) // 2
    if x == arr[mid]:
        return mid
    if x > arr[mid]:
        return bsearch(arr, x, mid + 1, last)
    return bsearch(arr, x, first, mid - 1)

```

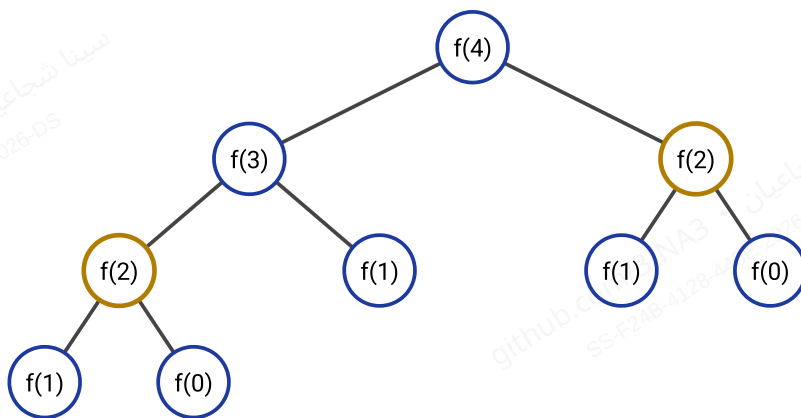
```
if __name__ == "__main__":
    print(fact(6), fib(7), add(3, 4), mul(3, 4), div(17, 5))    # 720 13 7 12 3
    print(is_palindrome("racecar"), bsearch([1, 3, 6, 7, 14], 7)) # True 3
    hanoi(2, "A", "B", "C")    # A to B / A to C / B to C
```

رشته‌ی فراخوانی (Call Stack) را ببینید

هر فراخوانی تا وقتی جواب زیرفراخوانی‌اش نرسیده «منتظر» می‌ماند و روی Call Stack می‌نشیند. به همین دلیل بازگشت هم LIFO است. ردگیری `fact(4)`:

برگشت (جواب‌ها)	رفت (فراخوانی‌ها)
<code>fact(4) = 4 * fact(3)</code>	<code>fact(1) = 1</code>
<code>fact(3) = 3 * fact(2)</code>	<code>fact(2) = 2 * 1 = 2</code>
<code>fact(2) = 2 * fact(1)</code>	<code>fact(3) = 3 * 2 = 6</code>
<code>fact(1) → شرط توقف</code>	<code>fact(4) = 4 * 6 = 24</code>

الگوی کلی: تمامی توابع این فصل یک قالب دارند: `if` شرط_توقف: `return` مقدار و `else: return` رابطه_بازگشتی. اگر شرط توقف نباشد یا هیچ‌وقت درست نشود، به `RecursionError` می‌رسید.



دوبار حساب می‌شود `f(2)`
(رنگ نارنجی)

درخت فراخوانی ۹: `fib(4)` فراخوانی و زیرمسئله‌های تکراری

چرا فیبوناچی کند است؟

در `fib(5)` مقدار `fib(3)` دو بار و `fib(2)` سه بار دوباره حساب می‌شود. تعداد فراخوانی‌ها برای $n=0..5$ برابر ۱، ۳، ۵، ۹، ۱۵ است و تقریباً با هر n ضرب در $1/6$ می‌شود $\Rightarrow O(2^n)$. راه‌حل (توضیح تکمیلی): نتیجه‌ها را یادداشت کنیم (Memoization) تا هر مقدار فقط یک بار حساب شود: $O(n)$.

```

def fib_memo(n: int, memo: dict[int, int] | None = None) → int:
    """حافظه 0(n) زمان و 0(n) فایبوناچی با حافظه‌ی نتایج."""
    memo = {} if memo is None else memo
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]

def sum_list_rec(arr: list[int]) → int:
    """جمع لیست: شرط توقف = لیست خالی، رابطه: اولی + جمع بقیه"""
    if not arr:
        return 0
    return arr[0] + sum_list_rec(arr[1:])

def power(a: int, b: int) → int:
    """a به توان b (b ≥ 0): P(a,b) = P(a,b-1) * a."""
    return 1 if b == 0 else power(a, b - 1) * a

def count_down(n: int) → None:
    """بعد از فراخوانی بود، ترتیب برعکس print فراخوانی بعد از چاپ: 1 2 3؛ اگر 0 می‌شد"""
    if n == 0:
        return
    print(n, end=" ")
    count_down(n - 1)

if __name__ == "__main__":
    print(fib_memo(40), sum_list_rec([1, 2, 3, 4]), power(2, 10)) # 102334155
    10 1024
    count_down(3) # 3 2 1

```

برج هانوی: ردگیری برای ۳ دیسک

ایده: برای جابه‌جایی n دیسک از A به C، ابتدا n-1 دیسک بالایی را به B ببر، بزرگ‌ترین را به C ببر، سپس n-1 دیسک را از B روی C بگذار. قوانین: هر بار یک دیسک، و دیسک بزرگ هیچ‌وقت روی کوچک‌تر نباشد. رابطه‌ی $T(n) = 2T(n-1) + 1$ حرکت (۳ دیسک: ۷ حرکت؛ ۱۰ دیسک: ۱۰۲۳).

خروجی hanoi(3, A, B, C):

- 1) A to C 2) A to B 3) C to B 4) A to C
- 5) B to A 6) B to C 7) A to C

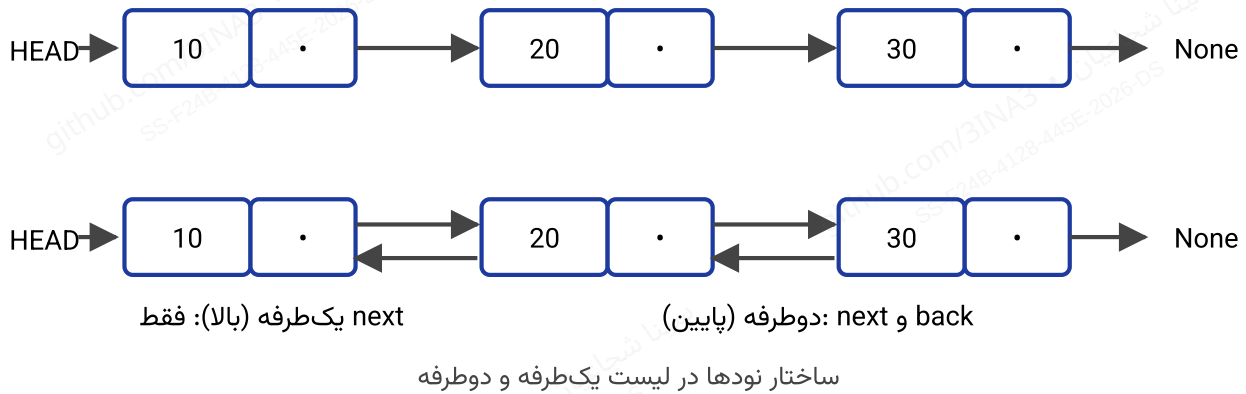
Python

```
def hanoi_moves(n: int, src: str = "A", aux: str = "B", dst: str = "C") →  
list[tuple[str, str]]:  
    """کلاس، ولی حرکته‌ها را برمی‌گرداند تا بشود شمرد/تست کرد همان hanoi"""  
    if n == 1:  
        return [(src, dst)]  
    return (hanoi_moves(n - 1, src, dst, aux)  
            + [(src, dst)]  
            + hanoi_moves(n - 1, aux, src, dst))  
  
if __name__ == "__main__":  
    moves = hanoi_moves(3)  
    print(len(moves), moves[:3])    # 7 [('A', 'C'), ('A', 'B'), ('C', 'B')]
```

تمرین: بدون اجرا بگویید (5, 17) div چند فراخوانی دارد؟ (پاسخ: ۴ فراخوانی با 2، 7، 12، 17 و جواب ۳)

۵. لیست پیوندی

مجموعه‌ای از نودها (داده + اشاره‌گر) که با head شروع می‌شوند. درج/حذف ابتدا $O(1)$ و جست‌وجو $O(n)$ است.



یک‌طرفه: HEAD → [10 | •] → [20 | •] → [30 | •] → None
حلقوی: HEAD → [10 | •] → [20 | •] → [30 | •] → HEAD
دوطرفه: None ⇌ [10] ⇌ [20] ⇌ [30] ⇌ None

یک‌طرفه

```

from typing import Any, Optional

class Node:
    def __init__(self, data: Any) → None:
        self.data, self.next = data, None    # type: ignore

class LinkedList:
    def __init__(self) → None:
        self.head: Optional[Node] = None

    def insert_first(self, x) → None:
        n = Node(x); n.next = self.head; self.head = n

    def insert_last(self, x) → None:
        n = Node(x)
        if not self.head:
            self.head = n; return
        t = self.head
        while t.next:
            t = t.next
        t.next = n

    def insert_after(self, x, data) → bool:
        t = self.head
        while t and t.data ≠ x:
            t = t.next
        if not t:
            return False
        n = Node(data); n.next = t.next; t.next = n
        return True

    def delete_first(self) → None:
        if self.head:
            self.head = self.head.next

    def delete_last(self) → None:
        if not self.head or not self.head.next:
            self.head = None; return
        t = self.head
        while t.next.next:
            t = t.next

```

```

t.next = None

def delete(self, x) → bool:
    if not self.head:
        return False
    if self.head.data == x:
        self.head = self.head.next; return True
    t = self.head
    while t.next and t.next.data ≠ x:
        t = t.next
    if not t.next:
        return False
    t.next = t.next.next
    return True

def replace(self, old, new) → bool:
    t = self.head
    while t and t.data ≠ old:
        t = t.next
    if t:
        t.data = new
    return t is not None

def show(self) → None:
    t, out = self.head, []
    while t:
        out.append(str(t.data)); t = t.next
    print(" → ".join(out) + " → None")

if __name__ == "__main__":
    L = LinkedList()
    for v in (20, 30): L.insert_last(v)
    L.insert_first(10); L.insert_after(20, 25); L.show()    # 10 → 20 → 25 → 30
    → None
    L.delete(25); L.delete_last(); L.show()                # 10 → 20 → None

```

حلقوی یک طرفه: نسخه‌ی کامل

در لیست حلقوی هیچ None ای وجود ندارد؛ نود آخر به head برمی‌گردد. پس شرط پایان پیمایش `t.next is not self.head` است (نه `t.next`)، و برای رسیدن به نود آخر باید یک بار کل لیست را پیماییم: درج/حذف انتها $O(n)$.


```
from typing import Any, Optional
```

```
class CNode:
```

```
    def __init__(self, data: Any) → None:
```

```
        self.data = data
```

```
        self.next: "CNode" = self          # تکنود به خودش اشاره می‌کند
```

```
class CLinkedList:
```

```
    """(head فقط) لیست پیوندی یکطرفه‌ی حلقوی."""
```

```
    def __init__(self) → None:
```

```
        self.head: Optional[CNode] = None
```

```
    def _last(self) → CNode:
```

```
        t = self.head
```

```
        while t.next is not self.head:
```

```
            t = t.next
```

```
        return t
```

```
    def _find(self, x: Any) → Optional[CNode]:
```

```
        if self.head is None:
```

```
            return None
```

```
        t = self.head
```

```
        while True:
```

```
            if t.data == x:
```

```
                return t
```

```
            t = t.next
```

```
            if t is self.head:
```

```
                # یک دور کامل شد
```

```
                return None
```

```
    def insert_first(self, x: Any) → None:
```

```
        a = CNode(x)
```

```
        if self.head is None:
```

```
            self.head = a
```

```
            return
```

```
        a.next = self.head
```

```
        self._last().next = a
```

```
        self.head = a
```

```
    def insert_last(self, x: Any) → None:
```

```
        a = CNode(x)
```

```

    if self.head is None:
        self.head = a
        return
    last = self._last()
    last.next = a
    a.next = self.head

def insert_after(self, x: Any, data: Any) → bool:
    t = self._find(x)
    if t is None:
        return False
    a = CNode(data)
    a.next = t.next
    t.next = a
    return True

def insert_mid(self, data: Any) → None:
    """(نود اول شماره‌ی 0) count//2 درج بعد از نود شماره‌ی 0."""
    if self.head is None or self.head.next is self.head:
        self.insert_first(data)
        return
    count, t = 1, self.head
    while t.next is not self.head:
        t = t.next
        count += 1
    t = self.head
    for _ in range(count // 2):
        t = t.next
    self.insert_after(t.data, data)

def delete_first(self) → None:
    if self.head is None:
        return
    if self.head.next is self.head:
        self.head = None
        return
    last = self._last()
    self.head = self.head.next
    last.next = self.head

def delete_last(self) → None:
    if self.head is None:
        return
    if self.head.next is self.head:
        self.head = None

```

```

        return
    t = self.head
    while t.next.next is not self.head:    # یک نود قبل از آخر
        t = t.next
    t.next = self.head

def delete_after(self, x: Any) → bool:
    t = self._find(x)
    if t is None or t.next is t:
        return False
    victim = t.next
    if victim is self.head:
        self.head = victim.next
    t.next = victim.next
    return True

def delete(self, x: Any) → bool:
    t = self._find(x)
    if t is None:
        return False
    if t is self.head:
        self.delete_first()
        return True
    p = self.head
    while p.next is not t:
        p = p.next
    p.next = t.next
    return True

def replace(self, old: Any, new: Any) → bool:
    t = self._find(old)
    if t is None:
        return False
    t.data = new
    return True

def show(self) → None:
    if self.head is None:
        print("empty")
        return
    t, out = self.head, []
    while True:
        out.append(str(t.data))
        t = t.next
        if t is self.head:

```

```

        break
    print(" → ".join(out) + " → (head)")

if __name__ == "__main__":
    c = CLinkedList()
    for v in (10, 20, 30): c.insert_last(v)
    c.insert_first(5); c.insert_after(20, 25); c.show()    # 5 → 10 → 20 → 25 -
→ 30 → (head)
    c.delete(25); c.delete_last(); c.delete_first(); c.show()    # 10 → 20 →
(head)

```

دو طرفه: نسخه‌ی کامل

هر نود دو اشاره‌گر دارد: `next` و `back`. مزیت: با داشتن خود نود می‌توان بدون پیمایش دوباره آن را حذف کرد (`t.back` همان «نود قبلی» است). باید **چهار اشاره‌گر** را در درج و حذف درست بروزرسانی کنید.

```

insert_after(x, d):  [x] ⇌ [y]    →    [x] ⇌ [new] ⇌ [y]
    ۱) new.next = x.next    ۲) new.back = x    ۳) y.back = new    ۴) x.next = new

```

```

class DNode:
    def __init__(self, data: Any) → None:
        self.back: Optional["DNode"] = None
        self.data = data
        self.next: Optional["DNode"] = None

class DLinkedList:
    """لیست پیوندی دوطرفه"""

    def __init__(self) → None:
        self.head: Optional[DNode] = None

    def _find(self, x: Any) → Optional[DNode]:
        t = self.head
        while t and t.data != x:
            t = t.next
        return t

    def insert_first(self, x: Any) → None:
        a = DNode(x)
        a.next = self.head
        if self.head:
            self.head.back = a
        self.head = a

    def insert_last(self, x: Any) → None:
        a = DNode(x)
        if self.head is None:
            self.head = a
            return
        t = self.head
        while t.next:
            t = t.next
        t.next = a
        a.back = t

    def insert_after(self, x: Any, data: Any) → bool:
        t = self._find(x)
        if t is None:
            return False
        a = DNode(data)
        a.next, a.back = t.next, t

```

```

    if t.next:
        t.next.back = a
    t.next = a
    return True

def delete_first(self) → None:
    if self.head is None:
        return
    self.head = self.head.next
    if self.head:
        self.head.back = None

def delete_last(self) → None:
    if self.head is None:
        return
    t = self.head
    while t.next:
        t = t.next
    if t.back:
        t.back.next = None
    else:
        self.head = None

def delete_after(self, x: Any) → bool:
    t = self._find(x)
    if t is None or t.next is None:
        return False
    victim = t.next
    t.next = victim.next
    if victim.next:
        victim.next.back = t
    return True

def delete(self, x: Any) → bool:
    t = self._find(x)
    if t is None:
        return False
    if t.back:
        t.back.next = t.next
    else:
        self.head = t.next
    if t.next:
        t.next.back = t.back
    return True

```

```

def replace(self, old: Any, new: Any) → bool:
    t = self._find(old)
    if t is None:
        return False
    t.data = new
    return True

def show(self) → None:
    t, out = self.head, []
    while t:
        out.append(str(t.data))
        t = t.next
    print(" ↔ ".join(out) if out else "empty")

if __name__ == "__main__":
    d = DLinkedList()
    for v in (20, 30): d.insert_last(v)
    d.insert_first(10); d.insert_after(20, 25); d.show()    # 10 ↔ 20 ↔ 25 ↔
30
    d.delete(25); d.delete_after(10); d.delete_last(); d.show()    # 10

```

دوطرفه‌ی حلقوی

ترکیب دو ایده: $last.next = head$ و $head.back = last$. نتیجه‌ی جالب: نود آخر همیشه $head.back$ است، پس $insert_last$ بدون پیمایش و در $O(1)$ انجام می‌شود.

```

class CDLinkedList:
    """لیست پیوندی دوطرفه‌ی حلقوی."""

    def __init__(self) → None:
        self.head: Optional[DNode] = None

    def _find(self, x: Any) → Optional[DNode]:
        if self.head is None:
            return None
        t = self.head
        while True:
            if t.data == x:
                return t
            t = t.next
            if t is self.head:
                return None

    def insert_last(self, x: Any) → None:      # 0(1)
        a = DNode(x)
        if self.head is None:
            a.next = a.back = a
            self.head = a
            return
        last = self.head.back
        last.next, a.back = a, last
        a.next, self.head.back = self.head, a

    def insert_first(self, x: Any) → None:
        self.insert_last(x)
        self.head = self.head.back          # نود تازه، سر می‌شود

    def insert_after(self, x: Any, data: Any) → bool:
        t = self._find(x)
        if t is None:
            return False
        a = DNode(data)
        a.next, a.back = t.next, t
        t.next.back = a
        t.next = a
        return True

    def _unlink(self, t: DNode) → None:
        if t.next is t:                      # تکنود

```

```

        self.head = None
        return
    t.back.next, t.next.back = t.next, t.back
    if t is self.head:
        self.head = t.next

def delete_first(self) → None:
    if self.head:
        self._unlink(self.head)

def delete_last(self) → None:
    if self.head:
        self._unlink(self.head.back)

def delete(self, x: Any) → bool:
    t = self._find(x)
    if t is None:
        return False
    self._unlink(t)
    return True

def delete_after(self, x: Any) → bool:
    t = self._find(x)
    if t is None or t.next is t:
        return False
    self._unlink(t.next)
    return True

def show(self) → None:
    if self.head is None:
        print("empty")
        return
    t, out = self.head, []
    while True:
        out.append(str(t.data))
        t = t.next
        if t is self.head:
            break
    print(" ↔ ".join(out) + " ↔ (head)")

if __name__ == "__main__":
    c = CDLinkedList()
    for v in (2, 3, 4): c.insert_last(v)

```

c.insert_first(1); c.show()

1 ↔ 2 ↔ 3 ↔ 4 ↔ (head)

c.delete_last(); c.delete(2); c.show()

1 ↔ 3 ↔ (head)

توضیح تکمیلی: در پایتون `del(self.head)` فقط نام را پاک می‌کند، نه شیء را؛ حافظه را زباله‌روب آزاد می‌کند.
لیست دوطرفه‌ی حلقوی همان دوطرفه است با `last.next = head` و `head.back = last`

عمل	یک طرفه	دوطرفه
insert_first	$O(1)$	$O(1)$
insert_last / delete_last	$O(n)$	$O(n)$
جست‌وجو / delete(x)	$O(n)$	$O(n)$

۶. درخت

درخت: یک ریشه و زیردرخت‌ها. **درجه‌ی نود** = تعداد فرزندان؛ **درخت دودویی** = درجه ۲؛ **ارتفاع** = طول طولانی‌ترین مسیر ریشه تا برگ.

نوع	تعریف	روابط (ارتفاع h از ۰)
پر (Full)	هر نود برگ یا درجه ۲	برگ‌ها = درجه‌دوها + ۱؛ نودها $2h+1 \leq$
کامل (Complete)	سطوح پر، آخرین سطح از چپ پر	نودها بین 2^h و $2^{h+1}-1$

13
/ \
11 15
/ \ / \
9 12 14 17

NLR (پیش‌ترتیب): 17 14 15 12 9 11 13
LNR (میان‌ترتیب): 17 15 14 13 12 11 9
LRN (پس‌ترتیب): 13 15 17 14 11 12 9
سطحی (BFS): 13 11 15 9 12 14 17

```
from collections import deque
```

```
class BNode:
```

```
    def __init__(self, d) → None:
        self.data, self.left, self.right = d, None, None
```

```
class BTree:
```

```
    def __init__(self, root: BNode | None = None) → None:
        self.root = root
```

```
    def nlr(self, n="root"):          # Preorder
        n = self.root if n == "root" else n
        return [] if not n else [n.data] + self.nlr(n.left) + self.nlr(n.right)
```

```
    def lnr(self, n="root"):          # Inorder
        n = self.root if n == "root" else n
        return [] if not n else self.lnr(n.left) + [n.data] + self.lnr(n.right)
```

```
    def lrn(self, n="root"):          # Postorder
        n = self.root if n == "root" else n
        return [] if not n else self.lrn(n.left) + self.lrn(n.right) + [n.data]
```

```
    def level_order(self) → list:
        out, q = [], deque([self.root] if self.root else [])
        while q:
            k = q.popleft(); out.append(k.data)
            q.extend(c for c in (k.left, k.right) if c)
        return out
```

```
class BST:
```

```
    """درخت جست و جوی دودویی: چپ > پدر > راست (بدون تکراری)"""
```

```
    def __init__(self) → None:
        self.root: BNode | None = None
```

```
    def add(self, x) → None:
        if not self.root:
            self.root = BNode(x); return
        t = self.root
        while True:
            if x == t.data:
```

```

        return
        side = "left" if x < t.data else "right"
        if getattr(t, side) is None:
            setattr(t, side, BNode(x)); return
        t = getattr(t, side)

def search(self, x) → bool:
    t = self.root
    while t and t.data ≠ x:
        t = t.left if x < t.data else t.right
    return t is not None

def inorder(self) → list:          همیشه مرتب شده
    return BTree(self.root).lnr()

if __name__ == "__main__":
    b = BST()
    for v in (13, 11, 15, 9, 12, 14, 17): b.add(v)
    print(b.inorder(), b.search(14))          # [9, 11, 12, 13, 14, 15, 17]
    True
    print(BTree(b.root).level_order())      # [13, 11, 15, 9, 12, 14, 17]

```

BST: حذف برگ ← قطع پیوند؛ حذف درجه یک ← پدر مستقیم به نوه وصل شود؛ حذف درجه دو ← جایگزینی با بزرگترین نود زیردرخت چپ (یا کوچکترین راست) و سپس حذف آن. جست‌وجو/درج: متوسط $O(\log n)$ ، بدترین $O(n)$.

هیپ (Heap)

درخت کامل که در لیست ذخیره می‌شود: فرزندان نود i در $2i+1$ و $2i+2$ ، پدر در $(i-1)//2$. در Max-Heap پدر از فرزندان بزرگتر است. درج: افزودن به انتها و حرکت به بالا؛ حذف ریشه: جابه‌جایی با آخری و حرکت به پایین. هر دو $O(\log n)$.

```

class MaxHeap:
    def __init__(self) → None:
        self.a: list = []

    def insert(self, x) → None:
        self.a.append(x)
        self._up(len(self.a) - 1)

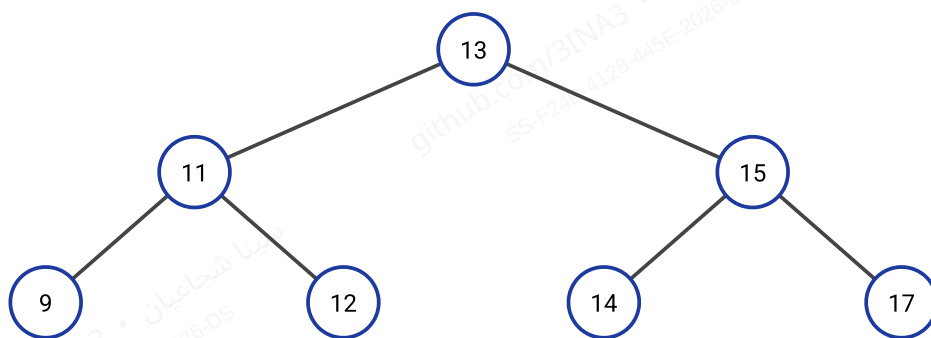
    def _up(self, n: int) → None:          # heapify down→up
        while n > 0:
            p = (n - 1) // 2
            if self.a[n] ≤ self.a[p]:
                break
            self.a[n], self.a[p] = self.a[p], self.a[n]
            n = p

    def _down(self, n: int) → None:        # heapify up→down
        while True:
            big, l, r = n, 2 * n + 1, 2 * n + 2
            if l < len(self.a) and self.a[l] > self.a[big]: big = l
            if r < len(self.a) and self.a[r] > self.a[big]: big = r
            if big == n:
                return
            self.a[n], self.a[big] = self.a[big], self.a[n]
            n = big

    def del_root(self):
        if not self.a:
            raise IndexError("heap is empty")
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last
            self._down(0)
        return top

if __name__ == "__main__":
    h = MaxHeap()
    for v in (10, 40, 15, 50, 100): h.insert(v)
    print([h.del_root() for _ in range(5)]) # [100, 50, 40, 15, 10]

```



NLR (پیش‌ترتیب): 17 14 15 12 9 11 13

LNR (میان‌ترتیب): 17 15 14 13 12 11 9

LRN (پس‌ترتیب): 13 15 17 14 11 12 9

سطحی (BFS): 13 11 15 9 12 14 17

یک درخت دودویی و چهار ترتیب پیمایش آن

اصطلاحات درخت با یک نگاه

معنا	اصطلاح
نود بدون فرزند (درجه ۰)	برگ
تعداد فرزندان / بیشینه‌ی درجه‌ی نودها	درجه‌ی نود / درخت
فاصله‌ی ریشه تا یک نود (ریشه سطح ۰) / فاصله‌ی ریشه تا عمیق‌ترین برگ	سطح و ارتفاع

درخت دودویی با افزودن و حذف

در این ساختار درخت با «درج در چپ‌ترین/راست‌ترین جا» ساخته می‌شود؛ ترتیب مقادارها اهمیتی ندارد (برخلاف BST). برای درج بعد از نود x : فرزند چپ قبلی x به فرزند چپ نود تازه تبدیل می‌شود، یعنی نود جدید «وسط» می‌نشیند. کلاس زیر از BTree بالا اثر می‌برد و پیمایش‌ها (nlr و ...) را آماده دارد.

```

class BTreeOps(BTree):
    """ BTree ارث‌بری از) درخت دودویی با حذف و حذف """

    def ins_left(self, d) → None:          # چپ‌ترین جا
        if self.root is None:
            self.root = BNode(d); return
        t = self.root
        while t.left:
            t = t.left
        t.left = BNode(d)

    def ins_right(self, d) → None:         # راست‌ترین جا
        if self.root is None:
            self.root = BNode(d); return
        t = self.root
        while t.right:
            t = t.right
        t.right = BNode(d)

    def _ins_after(self, node, x, d, side: str) → None:
        if node is None:
            return
        left, right = node.left, node.right    # قبل از تغییر ذخیره می‌کنیم
        if node.data == x:
            new = BNode(d)
            setattr(new, side, getattr(node, side))    # فرزند قبلی → نوه
            setattr(node, side, new)
        self._ins_after(left, x, d, side)
        self._ins_after(right, x, d, side)

    def ins_after_left(self, x, d) → None:
        self._ins_after(self.root, x, d, "left")

    def ins_after_right(self, x, d) → None:
        self._ins_after(self.root, x, d, "right")

    def del_left(self) → None:            # حذف چپ‌ترین نود
        if self.root is None:
            print("empty"); return
        if self.root.left is None:
            self.root = self.root.right; return
        p = self.root
        while p.left.left:

```

```

        p = p.left
        p.left = p.left.right          # زیردرخت راستِ نود حذفِ بالا می‌آید

def del_right(self) → None:          # حذفِ راست‌ترین نود
    if self.root is None:
        print("empty"); return
    if self.root.right is None:
        self.root = self.root.left; return
    p = self.root
    while p.right.right:
        p = p.right
    p.right = p.right.left

def delete_x(self, x) → bool:
    """را حذف می‌کند x زیردرختِ اولین نودِ دارای مقدار"""
    if self.root is None:
        return False
    if self.root.data == x:
        self.root = None
        return True
    return self._del_sub(self.root, x)

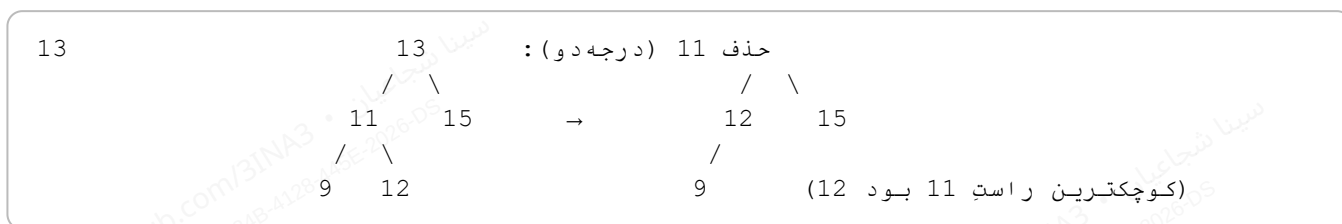
def _del_sub(self, node, x) → bool:
    for side in ("left", "right"):
        c = getattr(node, side)
        if c:
            if c.data == x:
                setattr(node, side, None)
                return True
            if self._del_sub(c, x):
                return True
    return False

if __name__ == "__main__":
    t = BTreeOps()
    for v in (1, 2, 3): t.ins_left(v)          # زنجیره‌ی چپ: 3 → 2 → 1
    t.ins_right(9); t.ins_after_left(1, 5)    # بین 1 و 2 می‌نشیند 5
    print(t.nlr(), t.level_order())          # [1, 5, 2, 3, 9] [1, 5, 9, 2, 3]
    t.del_left(); print(t.nlr())              # [1, 5, 2, 9]

```

BST: جست‌وجو، حذف و تبدیل لیست ↔ درخت

درج در BST همیشه «برگوار» است. حذف سه حالت دارد: ۱) برگ: پیوند پدر قطع شود. ۲) درجه یک: پدر مستقیم به نوه وصل شود. ۳) درجه دو: داده‌ی نود را با کوچکترین نودِ زیردرخت راست (یا بزرگترین چپ) عوض کن و آن نود را (که حداکثر یک فرزند دارد) حذف کن. تبدیل لیست به BST: اولین عنصر ریشه، بقیه به ترتیب درج می‌شوند؛ ترتیب درج شکل درخت را تعیین می‌کند ($[1,2,3,4] \rightarrow$ درخت کج و ارتفاع ۳، $[3,1,4,2] \rightarrow$ متعادل‌تر).



```

class BSTFull(BST):
    """BST کمینه/بیشینه، ارتفاع و تبدیل لیست با حذف."""

    def delete(self, x) → None:
        self.root = self._del(self.root, x)

    def _del(self, node, x):
        if node is None:
            return None
        if x < node.data:
            node.left = self._del(node.left, x)
        elif x > node.data:
            node.right = self._del(node.right, x)
        else:
            if node.left is None:           # برگ یا فقط فرزند راست
                return node.right
            if node.right is None:         # فقط فرزند چپ
                return node.left
            s = node.right                 # درجه دو: کوچکترین راست
            while s.left:
                s = s.left
            node.data = s.data
            node.right = self._del(node.right, s.data)
        return node

    def minimum(self):
        t = self.root
        while t and t.left:
            t = t.left
        return t.data if t else None

    def maximum(self):
        t = self.root
        while t and t.right:
            t = t.right
        return t.data if t else None

    def height(self, n="root") → int:
        n = self.root if n == "root" else n
        return -1 if n is None else 1 + max(self.height(n.left),
self.height(n.right))

    def descending(self) → list:           # کلاس: اول راست pshow مثل

```

```

out: list = []
def walk(n):
    if n:
        walk(n.right); out.append(n.data); walk(n.left)
walk(self.root)
return out

def create_tree(a: list) → BSTFull:
    t = BSTFull()
    for v in a:
        t.add(v)
    return t

def create_list(t: BSTFull) → list:    # ترتیب درج را برمی‌گرداند
    return list(t.list)

if __name__ == "__main__":
    t = create_tree([13, 11, 15, 9, 12, 14, 17])
    t.delete(11); print(t.inorder(), t.height())    # [9, 12, 13, 14, 15,
17] 2
    print(t.minimum(), t.maximum(), t.descending())    # 9 17 [17, 15, 14, 13,
12, 9]
    print(create_tree([1, 2, 3, 4]).height(), create_tree([3, 1, 4, 2]).height())
    # 3 2

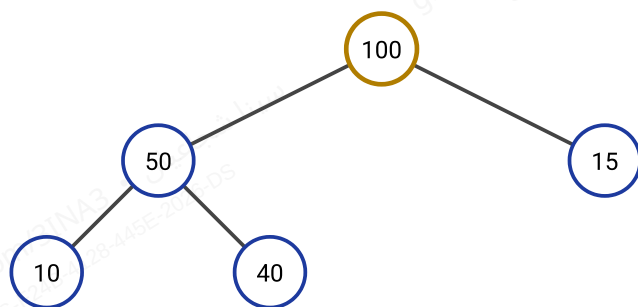
```

هیپ: ردگیری درج و حذف ریشه

درج ۱۰، ۴۰، ۱۵، ۵۰، ۱۰۰ در Max-Heap (هر عنصر با پدرش مقایسه و در صورت بزرگ‌تر بودن جابه‌جا می‌شود):

لیست بعد از heapify بالا	درج
[10]	10
[10, 40]	40
[15, 10, 40]	15
[10, 15, 40, 50]	50
[40, 10, 15, 50, 100]	100

حذف ریشه (۱۰۰): آخرین عنصر (۴۰) جای ریشه می‌نشیند و با بزرگ‌ترین فرزند (۵۰) جابه‌جا می‌شود $\Rightarrow$ ۵۰, ۴۰, ۱۵, ۱۰. هیپ فقط «پدر $\leq$ فرزند» را تضمین می‌کند؛ بین دو برادر ترتیبی نیست.



فرزندان i : $2i+1$ و $2i+2$

پدر: $(i-1)//2$

لیست:

100	50	15	10	40
0	1	2	3	4

Max-Heap: درخت کامل که در لیست ذخیره می‌شود

```

class MinHeap:
    """Min-Heap: پدر از فرزندان کوچکتر (یا مساوی) است."""

    def __init__(self) → None:
        self.a: list = []

    def insert(self, x) → None:
        self.a.append(x)
        self._up(len(self.a) - 1)

    def _up(self, n: int) → None:
        while n > 0:
            p = (n - 1) // 2
            if self.a[n] ≥ self.a[p]:
                break
            self.a[n], self.a[p] = self.a[p], self.a[n]
            n = p

    def _down(self, n: int) → None:
        while True:
            small, l, r = n, 2 * n + 1, 2 * n + 2
            if l < len(self.a) and self.a[l] < self.a[small]:
                small = l
            if r < len(self.a) and self.a[r] < self.a[small]:
                small = r
            if small == n:
                return
            self.a[n], self.a[small] = self.a[small], self.a[n]
            n = small

    def del_root(self):
        if not self.a:
            raise IndexError("heap is empty")
        top, last = self.a[0], self.a.pop()
        if self.a:
            self.a[0] = last
            self._down(0)
        return top

    def delete(self, value) → bool:
        """در هر دو جهت heapify حذف یک مقدار دلخواه: جایگزینی با آخری، سپس"""
        if value not in self.a:
            return False

```

```

        i, last = self.a.index(value), self.a.pop()
        if i < len(self.a):
            self.a[i] = last
            self._down(i)
            self._up(i)
        return True

def display(self) → None:
    print("heap:", self.a)

def heap_sort(values: list) → list:
    """توضیح تکمیلی: مرتب‌سازی با هیپ  $O(n \log n)$ ."""
    h = MinHeap()
    for v in values:
        h.insert(v)
    return [h.del_root() for _ in values]

if __name__ == "__main__":
    h = MinHeap()
    for v in (40, 10, 100, 15, 50): h.insert(v)
    h.display() # heap: [10, 15, 100, 40, 50]
    h.delete(15); h.display() # heap: [10, 40, 100, 50]
    print(heap_sort([5, 2, 9, 1, 7])) # [1, 2, 5, 7, 9]

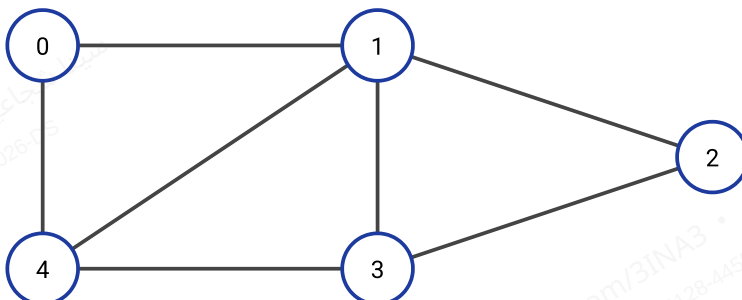
```

عمل	BST (متوسط / بدترین)	Heap
درج	$O(\log n) / O(n)$	$O(\log n)$
جست‌وجوی مقدار دلخواه	$O(\log n) / O(n)$	$O(n)$
دسترسی به min یا max	$O(\log n)$	$O(1)$

۷. گراف

گراف $G=(V,E)$: مجموعه‌ی رئوس و یال‌ها. با **ماتریس مجاورت** $n \times n$: خانه‌ی (s,t) یک است اگر یال باشد. در گراف وزن‌دار به جای ۱ وزن می‌نویسیم.

نوع	تعداد یال	درجه
بدون جهت	مجموع یک‌ها $\div 2 =$ مجموع درجه‌ها $\div 2$	تعداد یک‌های سطر
جهت‌دار	تعداد یک‌های ماتریس	یک‌های سطر + ستون؛ چاه: بدون خروجی، چشمه: بدون ورودی
کامل (n راس)	$n(n-1)/2$	هر راس $n-1$ ، قطر اصلی صفر



ماتریس مجاورت

	0	1	2	3	4
0	0	1	0	0	1
1	1	0	1	1	1
2	0	1	0	1	0
3	0	1	1	0	1
4	1	1	0	1	0

گراف بدون جهت با ۵ راس و ۷ یال و ماتریس مجاورت آن

تا عمق، سپس برگشت : (پشته/بازگشت) DFS سطح به سطح : (مف) BFS

0-1-2

از 0: 4 3 2 1 0

| \ |

(ترتیب به اندیس همسایه‌ها بستگی دارد)

4-3-1

```
from collections import deque
```

```
class Graph:
```

```
    """قابل تنظیم weighted و directed گراف با ماتریس مجاورت؛"""
```

```
    def __init__(self, n: int, directed: bool = False) → None:
```

```
        self.n, self.directed = n, directed
```

```
        self.m = [[0] * n for _ in range(n)]
```

```
    def _ok(self, *v: int) → bool:
```

```
        return all(0 ≤ i < self.n for i in v)
```

```
    def insert_edge(self, s: int, t: int, w: int = 1) → None:
```

```
        if self._ok(s, t):
```

```
            self.m[s][t] = w
```

```
            if not self.directed:
```

```
                self.m[t][s] = w
```

```
    def del_edge(self, s: int, t: int) → None:
```

```
        if self._ok(s, t):
```

```
            self.m[s][t] = 0
```

```
            if not self.directed:
```

```
                self.m[t][s] = 0
```

```
    def count_edges(self) → int:
```

```
        total = sum(1 for r in self.m for v in r if v)
```

```
        return total if self.directed else total // 2
```

```
    def bfs(self, start: int) → list[int]:
```

```
        seen, q, order = {start}, deque([start]), []
```

```
        while q:
```

```
            u = q.popleft(); order.append(u)
```

```
            for v in range(self.n):
```

```
                if self.m[u][v] and v not in seen:
```

```
                    seen.add(v); q.append(v)
```

```
        return order
```

```
    def dfs(self, start: int, seen: set | None = None) → list[int]:
```

```
        seen = set() if seen is None else seen
```

```
        seen.add(start); order = [start]
```

```
        for v in range(self.n):
```

```
            if self.m[start][v] and v not in seen:
```

```

        order += self.dfs(v, seen)
    return order

```

```

if __name__ == "__main__":
    g = Graph(5)
    for s, t in [(0, 1), (0, 4), (1, 2), (1, 3), (1, 4), (2, 3), (3, 4)]:
        g.insert_edge(s, t)
    print(g.count_edges(), g.bfs(0), g.dfs(0))    # 7 [0, 1, 4, 2, 3] [0, 1, 2, 3, 4]

```

توضیح تکمیلی: ماتریس مجاورت حافظه‌ی $O(V^2)$ و پیمایش $O(V^2)$ می‌گیرد؛ با لیست مجاورت (نسخه‌های آخر کد کلاس) حافظه $O(V+E)$ و پیمایش $O(V+E)$ می‌شود.

گراف جهت‌دار، وزن‌دار و محاسبه‌ی درجه‌ها

کلاس Graph بالا با `directed=True` جهت‌دار می‌شود و با `w` وزن می‌گیرد (وزن صفر یعنی «یال نیست»، پس وزن‌ها مثبت فرض می‌شوند). در جهت‌دار: **درجه‌ی خروجی** = یک‌های سطر، **درجه‌ی ورودی** = یک‌های ستون، و درجه‌ی کل = مجموع این دو. **چشمه** راسی بدون ورودی و **چاه** راسی بدون خروجی است.

```

1→2   4→2   1→0   2→0   2→3           0 1 2 3 4
[0 0 0 0 0] 0   (خروجی ندارند) 3 و 0 : چاه‌ها
[0 0 1 0 1] 1   (ورودی ندارند) 4 و 1 : چشمه‌ها
                                2 [1 0 0 1 0]   (ردیف 2: به 0 و 3)
                                3 [0 0 0 0 0]   4 [0 0 1 0 0]

```

```

def out_degree(g: Graph, u: int) → int:
    return sum(1 for v in range(g.n) if g.m[u][v])

def in_degree(g: Graph, u: int) → int:
    return sum(1 for r in range(g.n) if g.m[r][u])

def sources(g: Graph) → list[int]:
    return [u for u in range(g.n) if in_degree(g, u) == 0]

def sinks(g: Graph) → list[int]:
    return [u for u in range(g.n) if out_degree(g, u) == 0]

def complete_graph(n: int) → Graph:
    """n-1 یال، درجه‌ی هر رأس  $n(n-1)/2$ : گراف کامل بدون جهت"""
    g = Graph(n)
    for s in range(n):
        for t in range(s + 1, n):
            g.insert_edge(s, t)
    return g

if __name__ == "__main__":
    d = Graph(5, directed=True)
    for s, t in [(1, 2), (4, 2), (1, 0), (2, 0), (2, 3)]: d.insert_edge(s, t)
    print(sources(d), sinks(d), in_degree(d, 2), out_degree(d, 2))  # [1, 4] [0, 3] 2 2
    w = Graph(4)
    w.insert_edge(0, 1, 3); w.insert_edge(1, 2, 1); print(w.m[0][1],
w.count_edges())  # 3 2
    print(complete_graph(5).count_edges())  # 10

```

ردگیری DFS و BFS

خروجی	صف بعد از هر گام	BFS از ۰ (صف)
0	[4, 1]	برداشتن ۰
1 0	[3, 2, 4]	برداشتن ۱

4 1 0	[3, 2]	برداشتن ۴
3 2 4 1 0	[]	برداشتن ۲ و ۳

DFS از ۰: به اولین همسایه‌ی ملاقات نشده می‌رویم (۱)، از ۱ به ۲، از ۲ به ۳، از ۳ به ۴ $\Rightarrow$ DFS. 0 1 2 3 4. «عمق‌طلب» است و در واقع از پشته‌ی Call Stack استفاده می‌کند؛ BFS «سطح‌طلب» است و از صف. هر دو با ماتریس $O(V^2)$ و با لیست مجاورت $O(V+E)$.

لیست مجاورت: معمولی و پیوندی حلقوی

به جای ماتریس $V \times V$ ، برای هر راس فقط همسایه‌هایش را نگه می‌داریم؛ برای گراف‌های خلوت (E کوچک‌تر از V^2) حافظه‌ی $O(V+E)$ می‌گیرد. دو نسخه‌ی کلاس را با یک کلاس پایه به هم نزدیک کردیم: فقط روش نگهداری همسایه‌ها (`neighbors`) فرق دارد و BFS/DFS مشترک است.

```
from collections import deque
from typing import Iterator
```

```
class AdjGraph:
```

```
    """که زیرکلاس‌ها می‌دهند neighbors(u) روی DFS و BFS: کلاس پایه"""
```

```
    def __init__(self, n: int, directed: bool = True) → None:
        self.n, self.directed = n, directed
```

```
    def neighbors(self, u: int) → Iterator[tuple[int, int]]:
        raise NotImplementedError
```

```
    def bfs(self, start: int) → list[int]:
        seen, q, order = {start}, deque([start]), []
        while q:
            u = q.popleft(); order.append(u)
            for v, _ in self.neighbors(u):
                if v not in seen:
                    seen.add(v); q.append(v)
        return order
```

```
    def dfs(self, start: int, seen: set | None = None) → list[int]:
        seen = set() if seen is None else seen
        seen.add(start); order = [start]
        for v, _ in self.neighbors(start):
            if v not in seen:
                order += self.dfs(v, seen)
        return order
```

```
    def display(self) → None:
        for u in range(self.n):
            items = [f"{v}(w={w})" for v, w in self.neighbors(u)]
            print(f"{u}: {' '.join(items) if items else '-'}")
```

```
class ListGraph(AdjGraph):
```

```
    """لیست مجاورت پایتونی: adj[u] = [(v, w), ...]."""
```

```
    def __init__(self, n: int, directed: bool = True) → None:
        super().__init__(n, directed)
        self.adj: list[list[tuple[int, int]]] = [[] for _ in range(n)]
```

```

def insert_edge(self, s: int, t: int, w: int = 1) → None:
    if 0 ≤ s < self.n and 0 ≤ t < self.n:
        self.adj[s].append((t, w))
        if not self.directed:
            self.adj[t].append((s, w))

def del_edge(self, s: int, t: int) → None:
    if 0 ≤ s < self.n and 0 ≤ t < self.n:
        self.adj[s] = [x for x in self.adj[s] if x[0] ≠ t]
        if not self.directed:
            self.adj[t] = [x for x in self.adj[t] if x[0] ≠ s]

def neighbors(self, u: int) → Iterator[tuple[int, int]]:
    yield from self.adj[u]

```

```

class ANode:
    def __init__(self, v: int, w: int) → None:
        self.v, self.w = v, w
        self.next: "ANode" = self

```

```

class LinkedGraph(AdjGraph):
    """جهت دار و وزن دار با لیست پیوندی حلقوی برای هر راس"""

    def __init__(self, n: int) → None:
        super().__init__(n, True)
        self.adj: list[ANode | None] = [None] * n

    def insert_edge(self, s: int, t: int, w: int = 1) → None:
        if not (0 ≤ s < self.n and 0 ≤ t < self.n):
            return
        a, head = ANode(t, w), self.adj[s]
        if head is None:
            self.adj[s] = a
            return
        last = head
        while last.next is not head:
            last = last.next
        last.next, a.next = a, head

    def del_edge(self, s: int, t: int) → bool:
        head = self.adj[s]
        if head is None:
            return False

```

```

prev = head
while prev.next is not head:          # prev = آخرین نود
    prev = prev.next
cur = head
while True:
    if cur.v == t:
        if cur.next is cur:
            self.adj[s] = None
        else:
            prev.next = cur.next
            if cur is head:
                self.adj[s] = cur.next
            return True
    prev, cur = cur, cur.next
    if cur is head:
        return False

def neighbors(self, u: int) → Iterator[tuple[int, int]]:
    head = self.adj[u]
    if head is None:
        return
    t = head
    while True:
        yield t.v, t.w
        t = t.next
        if t is head:
            break

if __name__ == "__main__":
    for g in (ListGraph(5), LinkedGraph(5)):
        for s, t, w in [(0, 1, 2), (0, 4, 7), (1, 2, 1), (1, 3, 4), (2, 3, 5)]:
            g.insert_edge(s, t, w)
        g.del_edge(0, 4)
        print(g.bfs(0), g.dfs(0))          # [0, 1, 2, 3] [0, 1, 2, 3]
    g.display()                          # 0: 1(w=2) / 1: 2(w=1), 3(w=4) /
...

```

ویژگی	ماتریس مجاورت	لیست مجاورت
حافظه	$O(V^2)$	$O(V+E)$
آیا یال (s,t) هست؟	$O(1)$	$O(\text{درجه ی } s)$

$O(V+E)$	$O(V^2)$	BFS / DFS
گراف خلوت	گراف انبوه، پرسش‌های یال	مناسب برای

جدول مرور سریع و نکات امتحانی

نکته	زمان	ساختار / عمل
Underflow و Overflow؛ LIFO	$O(1)$	Stack push/pop/peek
FIFO؛ حلقوی با %	$O(1)$	Queue insert/delete
درج ابتدا $O(1)$	$O(n)$	Linked list جست‌وجو
درخت کج $\rightarrow$ بدترین حالت	$O(\log n) / O(n)$	BST جست‌وجو
ریشه همیشه min یا max	$O(\log n)$	Heap insert / delete root
صف / پشته	$O(V^2)$	BFS / DFS (ماتریس)

- همیشه خالی‌بودن را قبل از pop / delete / peek بررسی کنید.
- در لیست پیوندی، ترتیب بازنویسی اشاره‌گرها را حفظ کنید (اول new.next، بعد t.next).
- برای بازگشت: اول شرط توقف را بنویسید.